# Diseño e implementación de una solución distribuida en la nube para el procesamiento masivo de órdenes médicas en un sistema público de salud

Nicolás Meneses  Nicolás Mendoza

Universidad Pontificia Bolivariana

Proyecto COIL: Big Data y Metodología de la Investigación. Docente: Leonardo Betancur Agudelo

**Resumen.** Un sistema público de salud debe decidir, para cada orden médica, si el Estado cubre el servicio y qué porcentaje reconoce (100 %, 50 % o 25 % según el perfil del usuario), con trazabilidad de cada decisión y una meta de 12 millones de órdenes en menos de 24 horas. Se implementó una solución por lotes, distribuida y *serverless* en AWS: AWS Glue (Apache Spark) clasifica las órdenes; Amazon S3 guarda entradas y salidas en Parquet; Amazon Athena consulta la trazabilidad; AWS Secrets Manager custodia la clave con la que se seudonimiza al paciente (HMAC-SHA-256), y AWS CloudFormation aprovisiona todo. Sobre 1 200 000 órdenes, la lógica reproduce exactamente la referencia (780 000 cubiertas y 420 000 no cubiertas), conserva la igualdad entre entradas y salidas y no expone datos personales; las pruebas automatizadas cubren el 99,60 % de la lógica.

**Palabras clave:** big data, procesamiento por lotes, Apache Spark, computación serverless, salud pública, trazabilidad.

## 1. Planteamiento del problema

Un gobierno desea automatizar el análisis de órdenes médicas para decidir la cobertura pública, el porcentaje reconocido y la trazabilidad de cada decisión. Cada orden trae fecha, medio emisor, entidad emisora, valor del procedimiento y una glosa semiestructurada con el nombre, el identificador DNO y el seguro social del paciente, su edad, su perfil, el servicio solicitado y la medicación. La decisión depende de dos catálogos: 20 servicios cubiertos y tres perfiles (prioritario, 100 %; estándar, 50 %; copago, 25 %); si el servicio no está en el catálogo, la orden no se cubre.

Cada decisión es determinista; la dificultad está en tomarla a escala y con garantías. Procesar 12 millones de órdenes en 24 horas exige al menos 139 órdenes por segundo sostenidas; cada decisión debe poder auditarse, los datos personales no deben llegar a los resultados y el costo debe ser razonable para un presupuesto público.

**Pregunta de investigación.** ¿Puede una arquitectura por lotes, distribuida y serverless en AWS clasificar 12 millones de órdenes en menos de 24 horas, con trazabilidad individual, seudonimización de los pacientes y un costo cuantificable y razonable?

## 2. Objetivos

**General.** Diseñar, implementar y evaluar en AWS una solución por lotes, distribuida y serverless que clasifique la cobertura de 12 millones de órdenes en menos de 24 horas, con trazabilidad individual, seudonimización y un costo de operación cuantificado. **Específicos.** (1) Caracterizar los datos y formalizar las reglas de cobertura. (2) Diseñar la arquitectura justificando cada componente frente a sus alternativas. (3) Implementar el prototipo con infraestructura como código y pruebas automatizadas. (4) Evaluar tiempo, rendimiento, exactitud y trazabilidad con 1,2 y 12 millones de órdenes. (5) Estimar el costo con precios oficiales y consumo medido.

## 3. Estado del arte

MapReduce reparte el cómputo entre máquinas mediante funciones *map* y *reduce* (Dean y Ghemawat, 2008), y Apache Spark lo generalizó con un motor unificado en memoria (Zaharia et al., 2016). La arquitectura Lambda suma a la capa por lotes una de baja latencia (Marz y Warren, 2015), y la Kappa usa solo flujo continuo para no duplicar la lógica (Kreps, 2014); ambas atienden necesidades de baja latencia. Los lagos de datos con formatos columnares como Parquet permiten consultar los resultados leyendo solo las columnas necesarias (Armbrust et al., 2021). En reclamaciones médicas se usa el aprendizaje automático sobre todo para detectar fraude (Herland et al., 2018), pero el Reglamento (UE) 2024/1689 considera de alto riesgo la IA que decide el acceso a prestaciones públicas de salud (Parlamento Europeo y Consejo de la Unión Europea, 2024); con reglas explícitas, una decisión determinista que registra su regla es más explicable. Un hash simple de identificadores con formato conocido puede revertirse por fuerza bruta (AEPD y EDPS, 2019); HMAC lo impide con una clave secreta (Krawczyk et al., 1997). Los componentes están maduros; faltan implementaciones abiertas que reporten de forma verificable tiempo, exactitud, trazabilidad y costo.

## 4. Ruta elegida: procesamiento por lotes

Se eligió la opción 1, procesamiento por lotes, y no la ingesta continua e híbrida, por cuatro razones. (1) El reto fija una ventana de 24 horas y no pide respuesta inmediata por orden, así que la baja latencia no aporta a la meta. (2) Cada orden se decide de forma independiente, y una sola capa evita mantener la lógica en dos sistemas, la crítica de Kreps (2014) a la arquitectura Lambda. (3) Un servicio serverless cobra solo mientras procesa, sin ingesta encendida todo el día. (4) La trazabilidad se obtiene igual: cada decisión queda con el identificador de la ejecución y su marca de tiempo, consultable con SQL.

## 5. Arquitectura y método

Una plantilla de AWS CloudFormation crea todos los recursos de la Figura 1 y reutiliza el rol `LabRole`, porque el laboratorio no permite crear roles. Cada workflow lanza el mismo trabajo con un escenario: esqueleto (1 000 órdenes), muestra dañada, conjunto completo (1,2 M) y réplica (12 M: el conjunto diez veces, con identificadores únicos). Un quinto workflow, creado desactivado, programa la operación diaria.

El trabajo corre en AWS Glue 5.1, con Spark 3.5.6 y Python 3.11 (AWS, s.f.-c). Spark reparte las órdenes entre los workers y, en cada partición, aplica una lógica en Python puro, la misma que se prueba en local. Si el servicio está en el catálogo, asigna `cubre = SI`, el porcentaje del perfil y $\textit{valor cubierto} = \textit{valor} \times \textit{porcentaje}/100$ (regla `RG-01`); si no, `cubre = NO` y 0 % (`RG-00`). El paciente se identifica con

**Tabla 1:** Resultados frente a las cifras objetivo. La verificación local ejecuta la misma lógica en un solo proceso, sin Spark.

| Indicador | Cifra objetivo | Verificación local | Ejecución en AWS |
|---|---|---|---|
| Tiempo total, 12 M (H1) | $<$ 86 400 s (24 h) | No aplica | [s] con [W] workers G.1X |
| Rendimiento | $\geq$ 139 órdenes/s | 38 521 órdenes/s (1,2 M en 31,2 s) | 1,2 M: [órdenes/s]; 12 M: [órdenes/s] |
| Exactitud (H2) | 1,2 M: 780 000 SI y 420 000 NO; 12 M: 7 800 000 SI y 4 200 000 NO | 1,2 M: 780 000 SI y 420 000 NO (100 %) | 1,2 M: [SI], [NO]; 12 M: [SI], [NO] |
| Trazabilidad (H3) | Entradas = resultados + cuarentena | 1 200 000 = 1 200 000 + 0 | 12 M: [entradas] = [resultados] + [cuarentena] |
| Muestra dañada (34 órdenes) | 22 resultados; 12 en cuarentena (10 de formato, 2 de perfil) | 22; 12 (10 y 2) | [resultados]; [cuarentena] |
| Privacidad | 0 columnas con datos personales; clave fuera del código | 0 de 14 columnas; ninguna clave real en el código ni en la plantilla | Valor del secreto ausente de plantilla y registros: [sí/no] |
| Calidad del software | Cobertura de pruebas $\geq$ 80 % | 99,60 % (93 pruebas) | No aplica |
| Costo, ejecución de 12 M | Reportado con la fuente de cada precio | No aplica | [USD]; [USD] por orden |

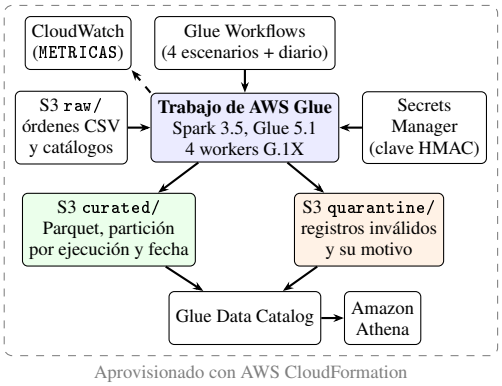



**Figura 1:** Arquitectura por lotes de la solución.

HMAC-SHA-256 de su DNO y seguro social. Las glosas mal formadas y los perfiles fuera de catálogo van a cuarentena con su motivo, sin detener el proceso, y antes de escribir el trabajo verifica que entradas = resultados + cuarentena. Cada resultado tiene 14 columnas, entre ellas la regla aplicada, el identificador de la ejecución y la marca de tiempo; ninguna contiene nombre, DNO ni seguro social.

El método es aplicado y cuantitativo, bajo la ciencia del diseño (Hevner et al., 2004) y las seis actividades de Peffers et al. (2007), con tres hipótesis: H1, 12 millones en menos de 24 horas; H2, coincidencia del 100 % con la referencia; H3, cada orden aparece una sola vez en resultados o en cuarentena. Se avanzó de forma incremental desde un esqueleto de extremo a extremo (Cockburn, 2004). La referencia proviene de un perfilado independiente: 1 200 000 órdenes sin duplicados en tres fechas; perfiles estándar (480 255), prioritario (419 653) y copago (300 092); 780 000 con servicio cubierto, 420 000 con alguno de 12 no cubiertos y ninguna glosa inválida. Por eso se construyó una muestra dañada de 34 órdenes: 20 válidas y 14 casos de error o formato irregular.

## 6. Resultados frente a las cifras objetivo

La Tabla 1 contrasta cada indicador con su cifra objetivo. El costo se modela como $C = C_{Glue} + C_{S3} + C_{Athena} + C_{CloudWatch} + C_{Secrets}$, con $C_{Glue} = W \times D \times t \times p$: $W$ workers, $D$ DPU por worker (1 en G.1X), $t$ horas facturadas por segundo y $p = 0{,}44$ USD por DPU-hora (AWS, s.f.-a). Athena cobra 5 USD por terabyte escaneado, volumen que Parquet reduce (AWS, s.f.-b), y el secreto cuesta 0,40 USD al mes (AWS, s.f.-d). Con el consumo medido, la ejecución de 12 millones costó [USD], y 30 ejecuciones diarias costarían [USD] al mes.

## 7. Conclusiones

La ruta por lotes con AWS Glue responde la pregunta de investigación con una arquitectura sencilla y reproducible desde una plantilla. La exactitud (H2) y la trazabilidad (H3) quedaron demostradas sobre las 1,2 millones de órdenes, y HMAC impide recalcular la identidad del paciente sin la clave. Con 12 millones, el tiempo total fue de [s], lo que [cumple / no cumple] la ventana de 24 horas (H1), con un costo de [USD] por ejecución. Las limitaciones son los datos sintéticos, cuya réplica conserva la distribución; una sola ejecución por escenario, y un rol de laboratorio con permisos amplios, que no permite restringir el acceso a la clave como en producción.

**Uso de inteligencia artificial.** Se usó un modelo de lenguaje (Claude, de Anthropic) como asistente en el diseño, el código y la redacción. Las referencias, las decisiones y la validación de resultados son responsabilidad de los autores.

## Referencias


Agencia Española de Protección de Datos y European Data Protection Supervisor. (2019). *Introduction to the hash function as a personal data pseudonymisation technique*. https://edps.europa.eu/sites/edp/files/publication/19-10-30_aepd-edps_paper_hash_final_en.pdf

Amazon Web Services. (s.f.-a). *AWS Glue pricing*. https://aws.amazon.com/glue/pricing/

Amazon Web Services. (s.f.-b). *Amazon Athena pricing*. https://aws.amazon.com/athena/pricing/

Amazon Web Services. (s.f.-c). *AWS Glue versions*. https://docs.aws.amazon.com/glue/latest/dg/release-notes.html

Amazon Web Services. (s.f.-d). *AWS Secrets Manager pricing*. https://aws.amazon.com/secrets-manager/pricing/

Armbrust, M., Ghodsi, A., Xin, R. y Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. En *CIDR '21*.

Cockburn, A. (2004). *Crystal clear: A human-powered methodology for small teams*. Addison-Wesley.

Dean, J. y Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM, 51*(1), 107–113. https://doi.org/10.1145/1327452.1327492

Herland, M., Khoshgoftaar, T. M. y Bauder, R. A. (2018). Big Data fraud detection using multiple Medicare data sources. *Journal of Big Data, 5*. https://doi.org/10.1186/s40537-018-0138-3

Hevner, A. R., March, S. T., Park, J. y Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–105. https://doi.org/10.2307/25148625

Krawczyk, H., Bellare, M. y Canetti, R. (1997). *HMAC: Keyed-hashing for message authentication* (RFC 2104). IETF. https://www.rfc-editor.org/rfc/rfc2104

Kreps, J. (2014, 2 de julio). Questioning the Lambda architecture. *O'Reilly Radar*.

Marz, N. y Warren, J. (2015). *Big data: Principles and best practices of scalable realtime data systems*. Manning.

Parlamento Europeo y Consejo de la Unión Europea. (2024). *Reglamento (UE) 2024/1689 [...] en materia de inteligencia artificial*. Diario Oficial de la Unión Europea.

Peffers, K., Tuunanen, T., Rothenberger, M. A. y Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A. et al. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM, 59*(11), 56–65. https://doi.org/10.1145/2934664